

Heaven's light is our guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report (Lab - 1)

Course Name : Software Engineering Sessional
Course Code : CSE 3206

Submitted by:

Kazi Md. Tamzid Shikto (2203007)
Humayra Islam Maysha (2203008)
Md. Rohanul Hasan Rijon (2203009)

Section: A
Session: 2022-2023

Submitted to:

Emrana Kabir Hashi
Assistant Professor,
Department of Computer Science & Engineering,
RUET

Name of the Experiment: Software Configuration Management with Git and GitHub

Introduction: Software Configuration Management (SCM) is essential for organizing, tracking and controlling changes throughout software development. In this experiment, Git and GitHub were used to manage the development of **ScoreSphere**, a web-based score tracking system for football matches, hackathons and chess tournaments. Git enabled version control through commits, branching and merging while GitHub supported collaboration among team members using remote repositories and pull requests. These tools helped maintain project consistency, simplify teamwork and ensure efficient management of the software development process.

Required Equipment/ Software:

- Computer or laptop with internet access.
- Git for version control.
- GitHub for remote repository hosting.
- Visual Studio Code (VS Code) for development.
- HTML, CSS and JavaScript for implementation.
- Google Chrome or Microsoft Edge for testing.

Procedure:

Step-1: Created the Project

Created a project folder named ScoreSphere and developed the web application using HTML, CSS and JavaScript in VS Code.

Step-2: Initialized the Local Repository

Opened Git Bash in the project directory and initialized a local Git repository.

```
git init
```

Step-3: Staged and Committed Project Files

Added all project files to the staging area and created the initial commit.

```
git add .  
git commit -m "Initial commit"
```

Step-4: Created a GitHub Repository

GitHub without initializing it with a README, license, or .gitignore file.

Step-5: Connected the Local Repository to GitHub

Connected the local Git repository with the remote GitHub repository.

```
git branch -M main
git remote add origin https://github.com/2203008-Humayra/scoreTracker
```

Step-6: Pushed the Project to GitHub

Pushed the local repository to the GitHub repository.

```
git push -u origin main
```

Step-7: Team Collaboration Using Feature Branches

After pushing the initial project to the main branch, team members created separate feature branches to implement different modules. One member developed the Hackathon module, while another implemented the Chess module. After completing their work, both members submitted Pull Requests to the main repository.

Step-8: Reviewed and Merged Pull Requests

Reviewed and merged the submitted Pull Requests into the main branch. Resolved merge conflicts whenever necessary to successfully integrate all modules into the project.

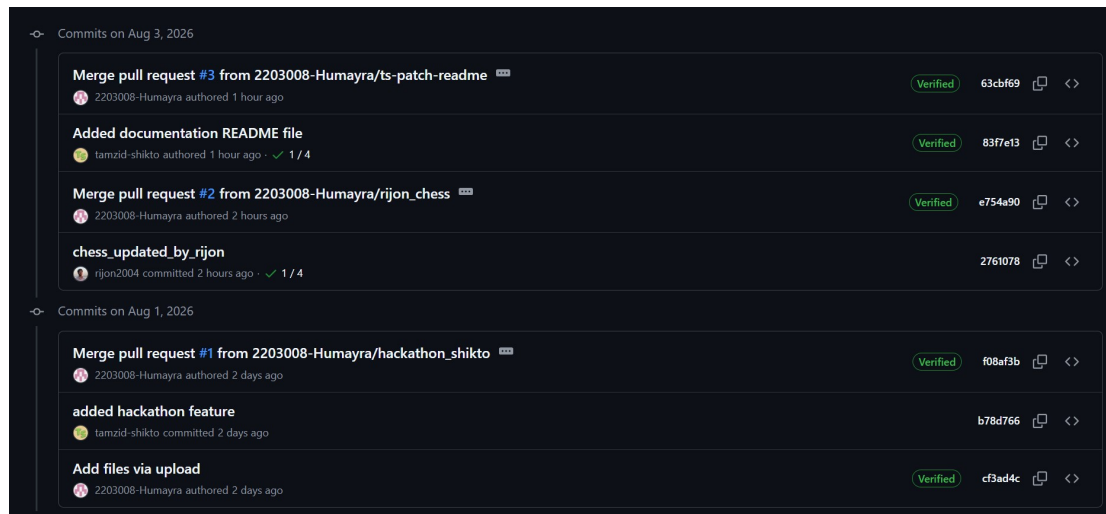
Step-9: Added Project Documentation

After merging the feature branches, a team member created a separate branch to add the project README and documentation files. The member then submitted a Pull Request, which was reviewed and merged into the main branch.

Step-10: Verified the Repository

Verified the GitHub repository to ensure that all source files, branches, commits, Pull Requests, README and documentation were successfully uploaded and synchronized.

Commits:



Precautions:

- Configured the correct Git username and email before making commits.
- Wrote clear and meaningful commit messages for every update.
- Created separate branches for new features to avoid affecting the main branch.
- Reviewed Pull Requests carefully before merging them into the main branch.
- Avoided committing sensitive information such as passwords or API keys to the repository.
- Verified all changes after merging to ensure the project remained functional.

Conclusion: This experiment provided practical experience in using Git and GitHub for managing a software development project. By applying version control techniques to the ScoreSphere project, the team was able to organize development tasks, collaborate through feature branches and integrate changes using Pull Requests. The workflow helped maintain a structured development process while reducing the risk of code conflicts. Overall, the experiment demonstrated the importance of Software Configuration Management in developing and maintaining collaborative software projects.